

Streams & Data Flow — Senior Developer Level

Streams are one of the most important abstractions for handling **large or continuous data without loading everything into memory at once**.

Instead of:

```
Entire Data
  ↓
Load into RAM
  ↓
Process
  ↓
Output
```

a stream uses:

```
Chunk → Process → Chunk → Process → Chunk → Process
```

This is especially useful for:

- Large files
- HTTP requests/responses
- File uploads/downloads
- Compression
- Encryption
- Database exports
- CSV/JSON processing
- Video/audio
- Network communication
- Data pipelines

1. What is a Stream?

A stream represents a sequence of data that becomes available **over time**.

For example, suppose you have a 5 GB file.

Without streams:

```
const data = await fs.promises.readFile("5gb-file.mp4");
```

Conceptually:

```
5 GB file
  ↓
RAM
  ↓
5 GB memory requirement
```

With a stream:

```
const stream = fs.createReadStream("5gb-file.mp4");
```

The file can be processed incrementally:

```
5 GB File
  ↓
Chunk 1 → process
Chunk 2 → process
Chunk 3 → process
Chunk 4 → process
...
```

Only a relatively small amount of data needs to be in memory at a time.

Stream Mental Model

Think of a stream like a **pipe carrying water**.

```
Source
  |
  | data
  ▼
[ Stream ]
  |
  | data
  ▼
Destination
```

A stream is not necessarily the data itself.

It is an abstraction for:

"Data is arriving or being consumed progressively."

Chunk

Stream data is usually handled in chunks.

For binary data in Node.js, chunks are commonly represented as `Buffer` objects.

Example:

Large file

Chunk 1	Chunk 2	Chunk 3	Chunk 4
---------	---------	---------	---------

Instead of:

```
const entireFile = ...
```

you process:

```
chunk1  
chunk2  
chunk3  
chunk4
```

Node.js Stream Types

Node.js provides four fundamental stream categories:

```
Readable  
Writable
```

Transform
Duplex

2. Readable Stream

A **Readable stream** is a source of data.

Examples:

- Reading a file
- HTTP request body
- Database result stream
- Incoming network data

Example:

```
import fs from "node:fs";

const readable = fs.createReadStream("./large.txt");

readable.on("data", (chunk) => {
  console.log(chunk);
});

readable.on("end", () => {
  console.log("Finished");
});
```

Conceptually:

```
File
|
▼
Readable Stream
|
├─ chunk 1
├─ chunk 2
├─ chunk 3
└─ chunk 4
```

Readable Stream Modes

A Node.js Readable can operate conceptually in two important modes:

Flowing mode

Data is automatically delivered through `'data'` events.

```
readable.on("data", (chunk) => {  
  console.log(chunk);  
});
```

The stream keeps producing data as it is consumed.

Paused mode

The stream waits until the consumer asks for data.

```
const chunk = readable.read();
```

This gives the consumer more direct control.

Modern Readable Consumption

Instead of manually using `'data'` events, async iteration is often cleaner:

```
for await (const chunk of readable) {  
  console.log(chunk);  
}
```

This is particularly useful with async processing.

Example:

```
for await (const chunk of readable) {  
  await processChunk(chunk);  
}
```

Readable Stream Events

Common events include:

```
data
end
error
close
readable
```

Example:

```
readable.on("error", (err) => {
  console.error(err);
});

readable.on("end", () => {
  console.log("No more data");
});
```

Important distinction:

```
'end'
```

means the stream has no more data to provide.

```
'close'
```

means the underlying resource has been closed.

They are related but not identical concepts.

3. Writable Stream

A **Writable stream** is a destination for data.

Examples:

- File
- HTTP response
- Socket
- Compression destination

Example:

```
import fs from "node:fs";

const writable = fs.createWriteStream("./output.txt");

writable.write("Hello\n");
writable.write("World\n");

writable.end();
```

Flow:

```
Application
  |
  | write()
  ▼
Writable Stream
  |
  ▼
File
```

write()

The most important method is:

```
writable.write(data);
```

But there is a critical detail:

`write()` returns a boolean.

```
const canContinue = writable.write(chunk);
```

This is directly connected to **backpressure**.

More on that shortly.

end()

When you're finished:

```
writable.end();
```

You can also provide final data:

```
writable.end("Final chunk");
```

Writable Events

Common events:

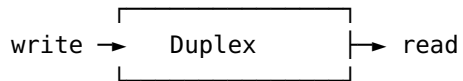
```
drain  
finish  
error  
close
```

For example:

```
writable.on("finish", () => {  
  console.log("All data has been flushed");  
});
```

4. Duplex Stream

A **Duplex stream** can both read and write.



In other words:

Readable + Writable

A network socket is a classic example.

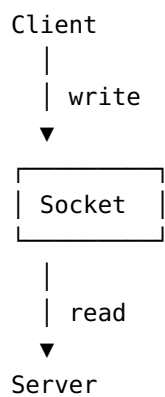
```
socket.write("Hello");
```

while also receiving data:

```
socket.on("data", (chunk) => {  
  console.log(chunk);  
});
```

Duplex Example

Think about a TCP connection:

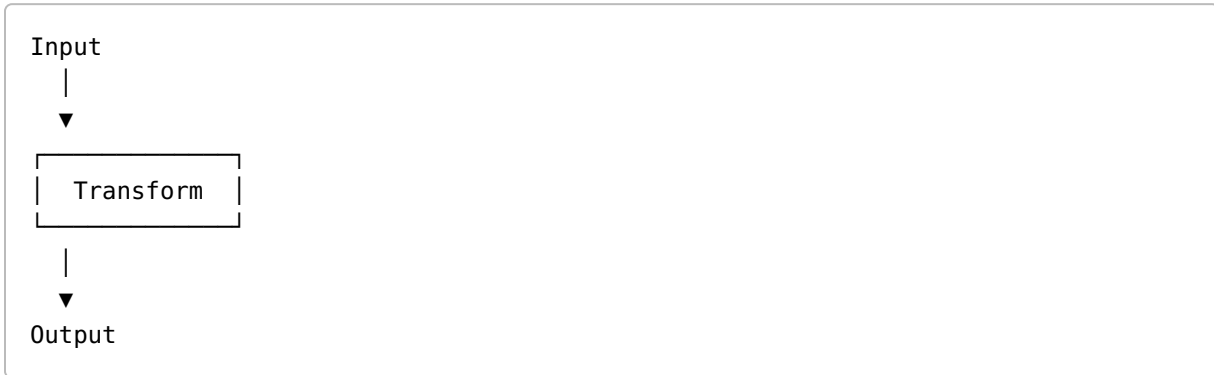


The socket is simultaneously:

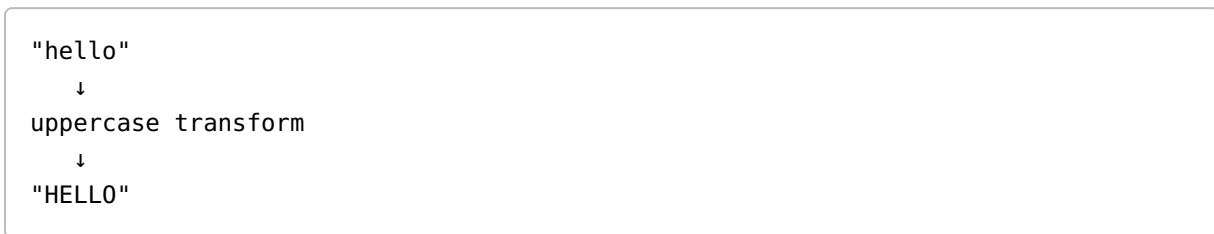
- Writable
- Readable

5. Transform Stream

A **Transform stream** is a special type of Duplex stream where the output is derived from the input.



Example:



Transform Example

```
import { Transform } from "node:stream";

const upperCase = new Transform({
  transform(chunk, encoding, callback) {
    const result = chunk.toString().toUpperCase();

    callback(null, result);
  }
});
```

Then:

```
process.stdin
  .pipe(upperCase)
  .pipe(process.stdout);
```

Flow:

```
stdin
  ↓
Readable
  ↓
uppercase Transform
  ↓
Writable
  ↓
stdout
```

Duplex vs Transform

This distinction is important.

Duplex

Input and output are independent.

```
Input → Duplex → Output
```

The output doesn't necessarily depend on the input.

Transform

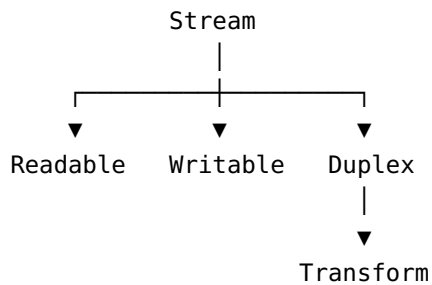
Output is generated from input.

```
Input
  ↓
Transform
  ↓
Modified Output
```

Therefore:

```
Transform ⊂ Duplex
```

Conceptually:



6. Why Streams Matter for Backend

Imagine an API that returns a 2 GB export.

Bad approach:

```
const data = await generateHugeExport();  
  
res.send(data);
```

Potential problem:

```
2 GB data  
↓  
RAM  
↓  
High memory pressure
```

Better:

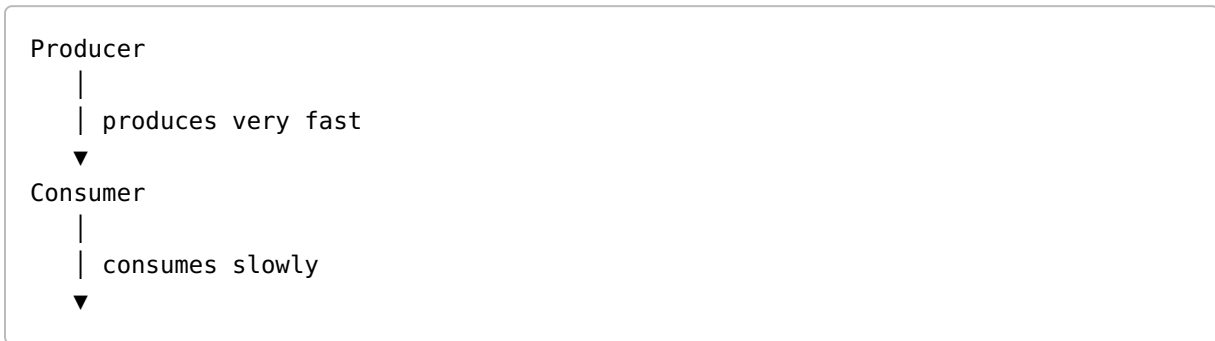
```
Database  
↓  
Readable Stream  
↓  
Transform  
↓  
HTTP Response
```

Data moves progressively.

7. Backpressure

Backpressure is one of the most important stream concepts.

Imagine:

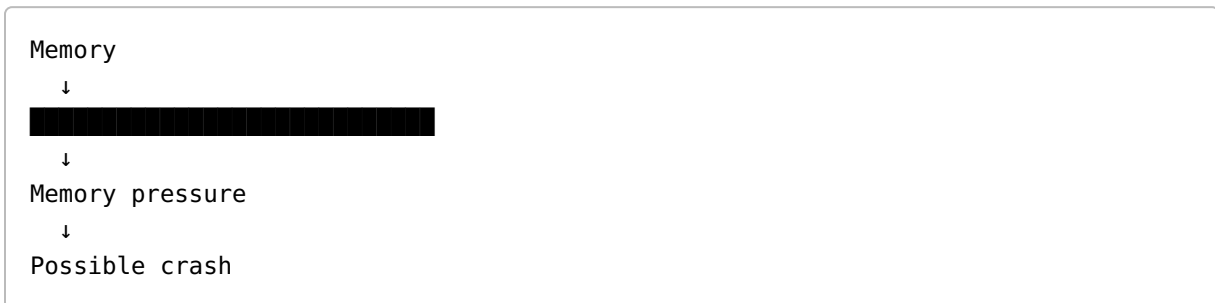


If the producer continuously produces data faster than the consumer can process it:



Data starts accumulating.

Eventually:



Backpressure is the mechanism that allows the consumer to effectively say:

"Slow down. I can't process data this quickly."

8. The Backpressure Problem

Suppose:

```
for (const chunk of hugeData) {  
  writable.write(chunk);  
}
```

This can be dangerous.

Why?

Because `write()` may return `false`.

```
const ok = writable.write(chunk);
```

If:

```
ok === false
```

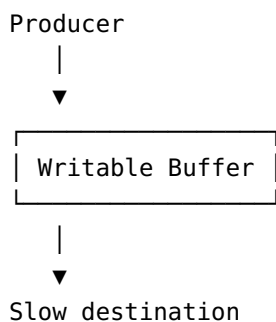
the writable stream is telling you:

"My internal buffer is getting full. Don't keep pushing blindly."

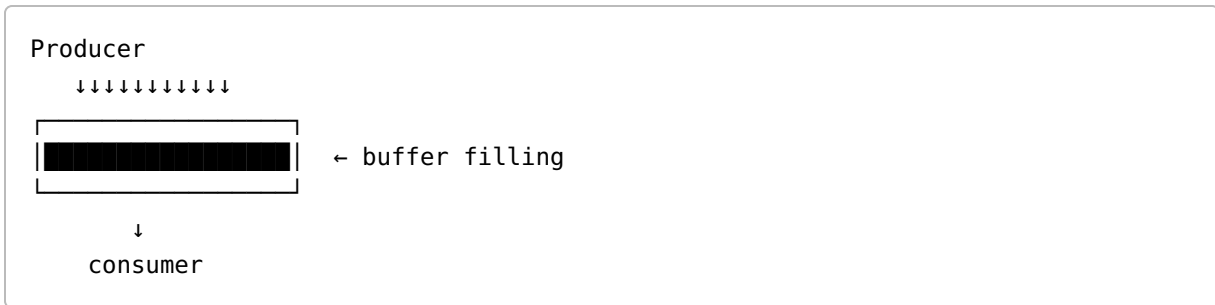
9. Writable Buffer

A Writable stream typically has an internal buffer.

Conceptually:



If the producer is faster:



Eventually the stream applies backpressure.

10. `write()` Return Value

Example:

```
if (!writable.write(chunk)) {  
  // Stop producing temporarily  
}
```

When the stream becomes ready again, it emits:

drain

Example:

```
if (!writable.write(chunk)) {  
  await once(writable, "drain");  
}
```

This is the basic idea of manual backpressure handling.

11. Backpressure with Async Iteration

For custom stream processing, you may encounter:

```
import { once } from "node:events";  
  
async function writeChunks(writable, chunks) {
```

```

    for (const chunk of chunks) {
      if (!writable.write(chunk)) {
        await once(writable, "drain");
      }
    }

    writable.end();
  }

```

Flow:

```

write()
├── true
│   └── ↓
│       continue
└── false
    └── ↓
        wait for drain
            ↓
            continue

```

This prevents the producer from overwhelming the Writable stream.

12. highWaterMark

Streams use buffering thresholds.

A major configuration is:

```
highWaterMark
```

Example:

```

const stream = fs.createReadStream("large.txt", {
  highWaterMark: 64 * 1024
});

```

Conceptually:

```
highWaterMark
  ↓
buffer management threshold
```

Important:

`highWaterMark` is **not necessarily a hard maximum memory limit**.

It is primarily a threshold used to determine when buffering should apply backpressure.

13. Backpressure Is End-to-End

This is a senior-level concept.

Suppose:

```
Database
  ↓
CSV Transform
  ↓
Compression
  ↓
HTTP Response
  ↓
Client
```

If the client is slow:

```
Client slow
  ↑
HTTP response buffer fills
  ↑
Compression slows
  ↑
Transform slows
  ↑
Database consumption slows
```

The pressure propagates backward.

That's the real power of a well-designed streaming pipeline.

14. pipe()

Node streams provide:

```
readable.pipe(writable);
```

Example:

```
import fs from "node:fs";

const source = fs.createReadStream("input.txt");
const destination = fs.createWriteStream("output.txt");

source.pipe(destination);
```

Flow:

```
input.txt
  ↓
Readable
  ↓
pipe()
  ↓
Writable
  ↓
output.txt
```

`pipe()` handles important flow-control behavior for you.

This is much safer than blindly calling `write()` for every chunk.

15. Transform Pipeline

Example:

```
File
  ↓
Readable
  ↓
```

```
Transform
↓
Compression
↓
Writable
```

Code:

```
source
  .pipe(transform)
  .pipe(compression)
  .pipe(destination);
```

This creates a data pipeline.

16. `pipeline()` — Safer Stream Chaining

`pipe()` is useful, but for production code, Node's:

```
pipeline()
```

is often preferable.

Import:

```
import { pipeline } from "node:stream/promises";
```

Example:

```
await pipeline(
  source,
  transform,
  destination
);
```

Conceptually:

```
Readable
↓
```

```
Transform
  ↓
Transform
  ↓
Writable
```

17. Why `pipeline()`?

A production stream pipeline has to handle:

- Errors
- Cleanup
- Premature termination
- Backpressure
- Stream destruction

`pipeline()` coordinates these pieces.

Example:

```
try {
  await pipeline(
    source,
    transform,
    destination
  );

  console.log("Pipeline completed");
} catch (error) {
  console.error("Pipeline failed:", error);
}
```

18. `pipe()` vs `pipeline()`

Feature	<code>pipe()</code>	<code>pipeline()</code>
Connect streams	✓	✓
Backpressure	✓	✓
Error handling	Manual	Better coordinated

Feature	<code>pipe()</code>	<code>pipeline()</code>
Cleanup	Less comprehensive	Better
Promise support	Not directly	<code>stream/promises</code>
Production pipelines	Useful	Usually preferred

The key idea:

`pipe()` connects streams; `pipeline()` manages a stream pipeline.

19. Real Production Example

Suppose users upload a large file.

Instead of:

```

Client
  ↓
Entire file in RAM
  ↓
Process
  ↓
Save

```

use:

```

Client
  ↓
HTTP Request
  ↓
Readable Stream
  ↓
Validation Transform
  ↓
Compression Transform
  ↓
File Writable
  ↓
Disk

```

Example conceptually:

```
await pipeline(
  req,
  validationTransform,
  compressionTransform,
  fs.createWriteStream(outputPath)
);
```

This is much more scalable for large data.

20. Error Propagation

Suppose:

```
Readable
  ↓
Transform A
  ↓
Transform B
  ↓
Writable
```

and Transform A fails.

A robust pipeline should ensure the pipeline doesn't simply leave other resources running indefinitely.

That's one of the reasons `pipeline()` is valuable.

Example:

```
try {
  await pipeline(
    source,
    transformA,
    transformB,
    destination
  );
} catch (err) {
  console.error(err);
}
```

21. Abortable Pipelines

Modern Node stream APIs can work with `AbortSignal`.

Example:

```
const controller = new AbortController();

await pipeline(
  source,
  transform,
  destination,
  {
    signal: controller.signal
  }
);
```

Later:

```
controller.abort();
```

This allows cancellation to propagate through the operation.

This connects directly with the earlier concept:

```
AbortController
  ↓
AbortSignal
  ↓
Pipeline cancellation
  ↓
Stream cleanup
```

22. Web Streams API

Node.js streams and Web Streams are related concepts, but they are **not identical APIs**.

The Web Streams API is a platform-standard streaming API used by browsers and also available in modern Node.js.

Main abstractions include:

```
ReadableStream
WritableStream
TransformStream
```

23. ReadableStream

A Web `ReadableStream` represents data that can be read incrementally.

Example:

```
const stream = new ReadableStream({
  start(controller) {
    controller.enqueue("Hello");
    controller.enqueue("World");

    controller.close();
  }
});
```

Conceptually:

```
ReadableStream
  |
  ├── "Hello"
  ├── "World"
  └── close
```

24. Reading a Web Stream

A consumer can use:

```
const reader = stream.getReader();

const { value, done } = await reader.read();
```

Repeatedly:

```
while (true) {  
  const { value, done } = await reader.read();  
  
  if (done) break;  
  
  console.log(value);  
}
```

But in many cases, async iteration provides a cleaner abstraction where supported:

```
for await (const chunk of stream) {  
  console.log(chunk);  
}
```

25. WritableStream

A Web `WritableStream` represents a destination.

Example:

```
const writable = new WritableStream({  
  write(chunk) {  
    console.log("Received:", chunk);  
  },  
  
  close() {  
    console.log("Finished");  
  }  
});
```

Conceptually:

```
Producer  
  ↓  
WritableStream  
  ↓  
write(chunk)
```

26. WritableStreamDefaultWriter

You can obtain a writer:

```
const writer = writable.getWriter();

await writer.write("Hello");
await writer.write("World");

await writer.close();
```

Important:

```
await writer.write(...)
```

can participate in flow control.

That is one way Web Streams expose backpressure to producers.

27. Web TransformStream

A Web `TransformStream` transforms data:

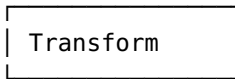
```
Readable
  ↓
TransformStream
  ↓
Writable
```

Example:

```
const transform = new TransformStream({
  transform(chunk, controller) {
    controller.enqueue(chunk.toUpperCase());
  }
});
```

The transform has two sides:

writable side



readable side

28. Web Streams and `fetch()`

This is where Web Streams become especially useful.

A response body can be a stream:

```
const response = await fetch(url);  
  
const body = response.body;
```

Instead of waiting for the entire response:

```
Server  
↓  
Huge response  
↓  
Entire body  
↓  
RAM
```

you can consume it incrementally.

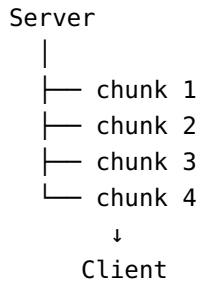
For example:

```
for await (const chunk of response.body) {  
  // Process chunk  
}
```

29. Streaming HTTP Response

A server can also produce data progressively.

Conceptually:



This is useful for:

- Large downloads
- Server-sent data
- AI-generated responses
- Video/audio
- Large JSON/CSV exports

30. Node.js Streams vs Web Streams

This distinction is extremely important.

Node.js Streams

Common classes:

```
Readable
Writable
Duplex
Transform
```

Imported from:

```
node:stream
```

They are deeply integrated into Node's APIs.

Web Streams

Common classes:

```
ReadableStream
WritableStream
TransformStream
```

They follow the Web Streams API model.

Comparison

Concept	Node.js	Web Streams
Readable	<code>Readable</code>	<code>ReadableStream</code>
Writable	<code>Writable</code>	<code>WritableStream</code>
Transform	<code>Transform</code>	<code>TransformStream</code>
Duplex	<code>Duplex</code>	No direct identical abstraction
Backpressure	Built in	Built in
<code>fetch()</code>	Can expose Web Streams	Native
Node filesystem	Native Node streams	Usually Node streams
Browser support	N/A	Native browser API

31. Converting Node Streams and Web Streams

Modern Node.js provides interoperability between the two stream models.

For example:

```
import { Readable } from "node:stream";

const webStream = Readable.toWeb(nodeReadable);
```

And:

```
const nodeStream = Readable.fromWeb(webReadable);
```

This is very useful when combining:

```
Web API
  ↓
Web Stream
  ↓
Node.js processing
  ↓
Node Writable
```

32. Stream Backpressure — Complete Mental Model

Suppose:

```
Producer
  ↓
Readable
  ↓
Transform
  ↓
Writable
  ↓
Consumer
```

Producer speed:



Consumer speed:



Without backpressure:

Buffer



With backpressure:

Producer



"Slow down"



Transform



Writable



Consumer

The system stabilizes around the consumer's capacity.

33. Streams Are About Flow Control, Not Just Memory

A common beginner understanding is:

"Streams save memory."

That's true, but incomplete.

Streams provide:

Streaming

+

Incremental processing

+

Backpressure

+

Flow control

+

Composition

+

Cancellation

+
Error propagation

Memory efficiency is only one benefit.

34. Example: Large CSV Processing

Suppose:

10 GB CSV

Bad:

```
const file = await fs.promises.readFile("data.csv");  
  
const rows = parse(file);
```

Potentially huge memory usage.

Better:

```
10 GB CSV  
  ↓  
Readable  
  ↓  
Line Parser  
  ↓  
Transform  
  ↓  
Database Writer
```

Only a portion of the file needs to be actively processed.

35. Example: File Compression

A streaming compression pipeline might look like:

```
large-file.txt
  ↓
Readable
  ↓
gzip Transform
  ↓
Writable
  ↓
large-file.txt.gz
```

Conceptually:

```
await pipeline(
  fs.createReadStream("large-file.txt"),
  createGzip(),
  fs.createWriteStream("large-file.txt.gz")
);
```

No need to load the entire file into memory first.

36. Example: HTTP Proxy

Imagine:

```
Client
  ↓
Your Node Server
  ↓
Upstream API
```

Instead of:

```
Upstream
  ↓
Entire response
  ↓
RAM
  ↓
Client
```

you can stream:

```
Upstream
  ↓
Readable
  ↓
HTTP Response
  ↓
Client
```

This is particularly useful for large responses.

37. Stream Error Handling

Never assume:

```
stream.pipe(otherStream);
```

means the whole operation is automatically safe.

Production code needs deliberate handling of:

- `error`
- cancellation
- resource cleanup
- premature connection close
- downstream failure

For multi-stage pipelines, prefer:

```
await pipeline(...)
```

when appropriate.

38. Common Stream Mistakes

Mistake 1 — Loading everything into memory

```
const data = await fs.promises.readFile(file);
```

For small files this is perfectly fine.

For huge files, consider streaming.

Mistake 2 — Ignoring backpressure

Bad:

```
for (const chunk of chunks) {  
  writable.write(chunk);  
}
```

Better:

```
for (const chunk of chunks) {  
  if (!writable.write(chunk)) {  
    await once(writable, "drain");  
  }  
}
```

Or use a properly designed pipeline.

Mistake 3 — Treating `highWaterMark` as a hard memory limit

It isn't.

It is primarily a buffering/backpressure threshold.

Mistake 4 — Confusing Duplex and Transform

Duplex:

```
read + write
```

Transform:

```
input → transformation → output
```

Mistake 5 — Mixing Node and Web stream APIs blindly

These are related but different abstractions:

Readable

vs:

ReadableStream

Know which API you are using.

39. `pipe()` vs `pipeline()` — Senior Rule

A useful rule:

```
Simple local stream connection
  ↓
  pipe()
```

For a serious multi-stage production pipeline:

```
multiple stages
+
error handling
+
cleanup
+
cancellation
  ↓
  pipeline()
```

Don't blindly use `pipe()` everywhere just because it is shorter.

40. Streams and Async Iteration

Streams integrate naturally with async iteration.

Example:

```
for await (const chunk of readable) {  
  await process(chunk);  
}
```

This creates a very readable processing model:

```
chunk  
↓  
await process()  
↓  
next chunk  
↓  
await process()  
↓  
next chunk
```

This can naturally slow consumption when processing is asynchronous.

41. Streams + AbortController

Modern applications often combine:

```
Stream  
+  
AbortController  
+  
AbortSignal
```

Example architecture:

```
HTTP Request  
↓  
Readable  
↓  
Transform  
↓  
Writable
```

↑
AbortSignal

If the user disconnects or an operation is cancelled:

```
abort()  
  ↓  
pipeline stops  
  ↓  
resources cleaned
```

This is an important production pattern.

42. Stream vs Buffer

These concepts are different.

Buffer

Represents data already held in memory.

```
const buffer = Buffer.from("Hello");
```

Stream

Represents a mechanism for moving/processing data progressively.

```
Stream  
  ↓  
chunk  
  ↓  
chunk  
  ↓  
chunk
```

A stream may deliver data as Buffers, strings, objects, or other values depending on its configuration.

43. Stream vs Promise

A Promise represents:

One eventual result

A Stream represents:

Many values over time

Conceptually:

Promise:

start → result

Stream:

start → chunk → chunk → chunk → chunk → end

That's why a 10 GB file is a better candidate for a stream than a single Promise containing the entire file.

44. Stream vs EventEmitter

A Node stream uses event-based behavior, but a stream provides much more than a generic EventEmitter.

It has semantics around:

- Reading
- Writing
- Buffering
- Backpressure
- Ending
- Errors
- Piping

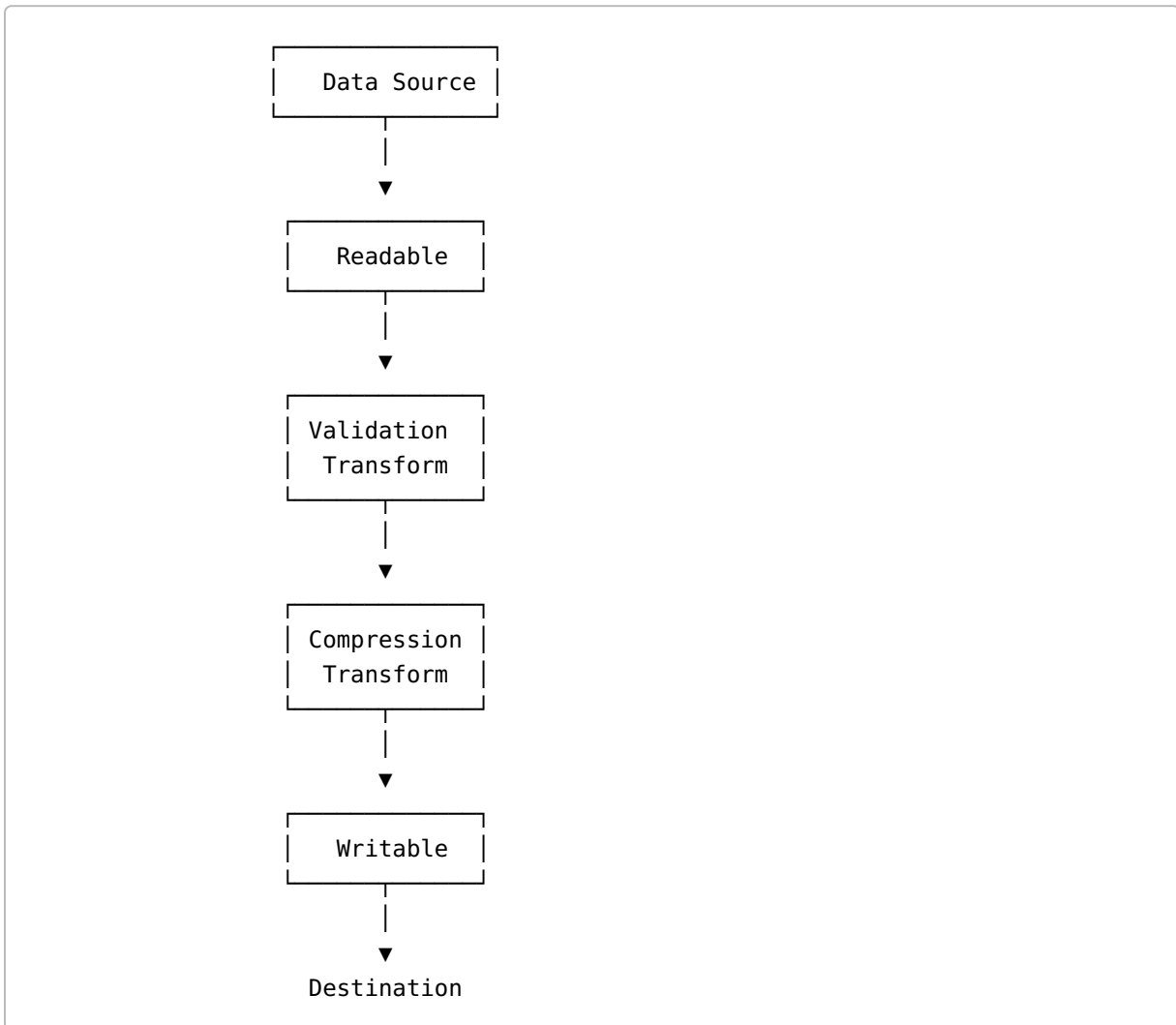
So:

Stream

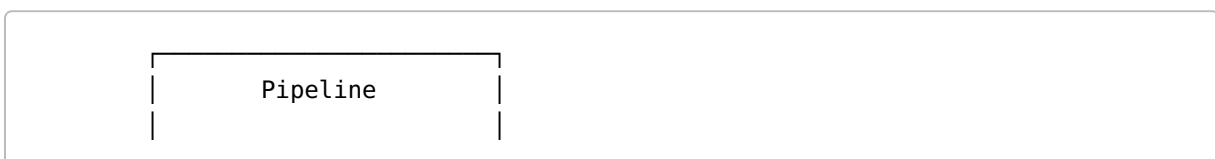
is a specialized abstraction built around data flow.

45. Production Architecture

A common backend architecture:



And around the whole thing:



Backpressure
Error propagation
Cancellation
Resource cleanup

46. Senior-Level Mental Model

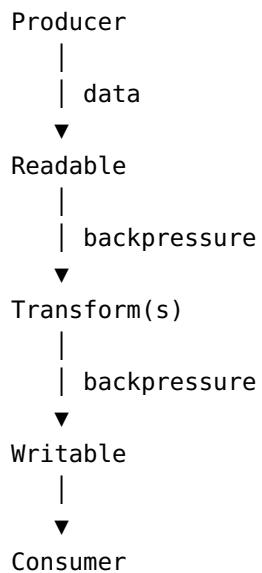
Don't think:

"Stream = reading files in chunks."

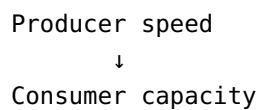
Think:

A stream is a controlled data-flow abstraction that allows producers and consumers to operate incrementally, while coordinating buffering, backpressure, errors, and completion.

The deeper architecture is:



The key relationship is:



↓
Backpressure
↓
Buffer control
↓
Stable memory usage

47. Quick Revision

Readable

Source → application

Example:

```
fs.createReadStream()
```

Writable

Application → destination

Example:

```
fs.createWriteStream()
```

Duplex

Read + Write

Example:

Socket

Transform

Input → modify → Output

Example:

```
gzip
```

Backpressure

```
Producer faster than consumer
  ↓
Buffer fills
  ↓
Slow producer
  ↓
Consumer catches up
```

```
pipe()
```

```
readable.pipe(writable);
```

Connects streams and handles flow control.

```
pipeline()
```

```
await pipeline(a, b, c);
```

Better abstraction for robust multi-stage pipelines.

Web Streams

```
ReadableStream
WritableStream
TransformStream
```

Platform-standard streaming abstraction.

48. Interview Questions

Beginner

1. What is a Node.js stream?

2. Why are streams useful for large files?
3. What is a Readable stream?
4. What is a Writable stream?
5. What is a Duplex stream?
6. What is a Transform stream?

Intermediate

1. What is backpressure?
2. What does `write()` returning `false` mean?
3. What is `drain`?
4. What is `highWaterMark`?
5. How does `pipe()` handle backpressure?
6. Why might `pipeline()` be preferable to manually chaining `pipe()`?
7. What's the difference between `end` and `finish`?
8. What's the difference between a Duplex and Transform stream?

Senior

1. Why does backpressure matter in distributed systems?
2. How does backpressure propagate through multiple transforms?
3. Why doesn't `highWaterMark` represent a strict memory limit?
4. How would you stream a 20 GB file through an HTTP API?
5. How would you safely cancel a long-running stream pipeline?
6. What happens if a downstream Writable becomes slower than the upstream producer?
7. How are Node.js streams different from Web Streams?
8. How would you bridge a Web `ReadableStream` to a Node `Readable`?
9. When would buffering be preferable to streaming?
10. How would you design a streaming CSV → database ingestion pipeline?

49. Practice Tasks

Task 1 — File Copy

Build:

```
input.txt
  ↓
Readable
  ↓
Writable
  ↓
output.txt
```

without `readFile()`.

Task 2 — Uppercase Transform

Create:

```
stdin
↓
uppercase transform
↓
stdout
```

Task 3 — Backpressure

Write a custom producer that generates data faster than a Writable consumes it.

Observe:

```
write() === false
```

and wait for:

```
drain
```

Task 4 — Compression Pipeline

Build:

```
large.txt
↓
Readable
↓
gzip
↓
Writable
```

using:

```
pipeline()
```

Task 5 — HTTP Streaming

Create an HTTP endpoint:

```
GET /download
```

that streams a large file directly to the client.

Don't load the whole file into RAM.

Task 6 — Transform Pipeline

Build:

```
File
↓
Line parser
↓
Filter
↓
Transform
↓
Output
```

For example:

```
10 GB CSV
↓
keep only active users
↓
convert format
↓
output.csv
```

Task 7 — Web Stream

Create a:

```
ReadableStream
```

that emits numbers:

```
1  
2  
3  
4  
5
```

then consume it using async iteration.

Final Takeaway

The four concepts you should permanently remember are:

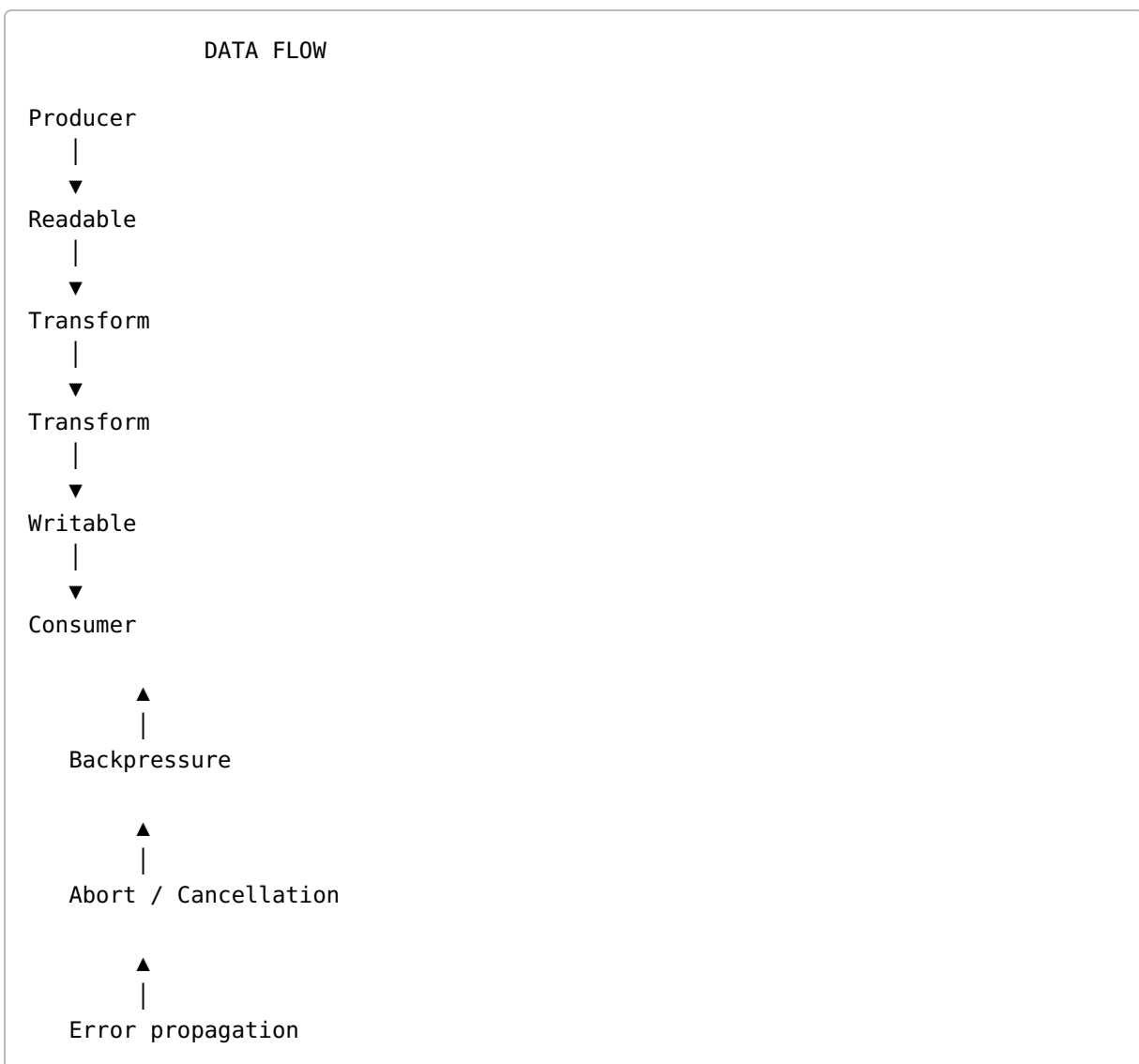
```
Readable  
  ↓  
Where data comes from  
  
Writable  
  ↓  
Where data goes  
  
Transform  
  ↓  
How data changes while flowing  
  
Backpressure  
  ↓  
How the system prevents a fast producer  
from overwhelming a slow consumer
```

Then:

```
Readable
  ↓
Transform
  ↓
Transform
  ↓
Writable
```

becomes a **stream pipeline**.

For production Node.js applications, the important mental model is:



Once you understand **data flow + backpressure + pipeline + cancellation**, Node.js streams stop looking like a collection of APIs and start looking like a coherent system.